LEKÒL KONEKTE

Documentation technique

Plateforme de gestion scolaire

Inscriptions · Paiements · Notes · Bulletins · Documents officiels

Créée et développée par **Plaisival Steven**
https://lekolkonekte.online · https://plaisivalsteven.online
Version du document : 4 octobre 2026

Table des matières

1. Présentation
2. Architecture technique
3. Structure du projet & volumétrie
4. Rôles et comptes
5. Authentification et sécurité d'accès
6. Modules fonctionnels
7. Contrôle d'accès du propriétaire
8. Base de données
9. API REST
10. Sécurité & tests d'intrusion
11. Déploiement
12. Tests et qualité
13. Crédits

1. Présentation

Lekòl Konekte est une plateforme web de gestion scolaire conçue pour les écoles haïtiennes. Elle relie les familles, les élèves et l'administration : inscriptions en ligne, paiements mobiles et par carte, saisie et publication des notes, bulletins PDF au format traditionnel haïtien, documents officiels, présence par badge QR, et plusieurs espaces publics gérés indépendamment (vlog, académie, corps enseignant, à propos, examens passés).

Devise du projet : « *Edikasyon ak Koneksyon pou demen* ».

Points forts

- Inscription en ligne avec paiement des frais préalable (activable/désactivable par le propriétaire).
- Paiements mobiles haïtiens (MonCash, NatCash, KashPay) et carte Visa/Mastercard.
- Bulletins PDF au format traditionnel haïtien, avec logo de l'école et appréciation rédigée par IA.
- Triple authentification possible : mot de passe + 2FA courriel, connexion biométrique (Face ID / Touch ID / Windows Hello) par passkeys, ou Google.
- Auto-inscription des élèves : un élève peut créer son compte et s'inscrire (si le propriétaire l'autorise).
- Assistant IA pour le personnel et tuteur IA pour les élèves.
- Présence quotidienne par scan du badge QR, avec notification au parent.
- Rapport mensuel d'activité (direction/secrétariat) exportable en Excel/CSV.
- Sécurité renforcée : compte base de données à privilèges limités, journal de sécurité, alertes, en-têtes stricts — validée par un test d'intrusion.

2. Architecture technique

Couche	Technologie
Frontend	React 18 + Vite, React Router 7, FontAwesome, jsQR / qrcode.react (badges QR), @simplewebauthn/browser
Backend	Node.js + Express (API REST, modules ES), sessions MySQL
Base de données	MySQL (≈38 tables)
Authentification	Sessions serveur (cookie signé), 2FA courriel, WebAuthn / passkeys, Google Identity
Paiements	Agrégateur mobile haïtien (plopplop) : MonCash, NatCash, KashPay, carte
Courriel / SMS	API Resend (HTTPS) / Textbelt (HTTPS)
IA	Groq (API compatible OpenAI), clés strictement côté serveur
PDF	pdf-lib (bulletins, relevés, duplicatas)
Déploiement	Railway (mono-service Docker), volume persistant, compte MySQL applicatif à privilèges limités

> Le frontend compilé est servi par le même serveur Express : une seule origine, donc les cookies de session fonctionnent sans configuration cross-site. Le serveur s'exécute sous l'utilisateur `node` (pas root).

3. Structure du projet & volumétrie

```
backend/
  src/config/        connexions (db, mailer, sms, plopplop, ia, webauthn, google, upload)
  src/controllers/   logique des routes
  src/models/        accès MySQL (requêtes paramétrées)
  src/services/      logique métier (paiement, bulletin, publication, sécurité, rapport…)
  src/middleware/    auth, rôles, rate-limit, protection anti-force brute
  src/routes/        modules de routes (≈188 endpoints)
  db/                schéma complet + 39 migrations incrémentales
  scripts/           init DB, seeds, sauvegarde
  test/              tests (node:test)
frontend/
  src/pages/         pages par espace (public, auth, parent, admin, direction, propriétaire…)
  src/components/    composants réutilisables
  src/context/       AuthContext, MarqueContext
  src/services/      api.js, biometrie.js
  index.html         métadonnées SEO + données structurées schema.org (auteur)
docs/                spécifications, rapport de sécurité, cette documentation
Dockerfile, railway.json
```

Partie	Lignes de code
Backend — code (src)	≈ 11 300
Backend — base de données (schéma + migrations)	≈ 1 100
Backend — tests	≈ 780
Frontend — code + styles	≈ 14 200
Total (code écrit, hors bibliothèques)	**≈ 27 400 lignes**

4. Rôles et comptes

Comptes principaux (table `utilisateurs`)

Rôle	Accès
parent	Inscriptions, paiements, notes/bulletins, documents officiels
admin / direction (secrétariat)	File des inscriptions, saisie et validation des notes, gestion
propriétaire	Contrôle d'accès, statistiques, permissions, annonces groupées, rapport mensuel
professeur	Devoirs, documents de cours, horaires pour ses classes

Un **compte élève** (auto-inscription) utilise le même modèle qu'un compte parent mais est marqué (`compte_eleve`) : l'interface affiche « Espace élève » et remplace les libellés « parent » par « responsable ».

Espaces indépendants (authentification séparée)

Économat, Vlog, À propos, Académie, Examens passés, Corps enseignant — chacun avec sa propre connexion et ses comptes administrateurs.

Espace élève par code

Connexion par code d'accès (sans courriel ni mot de passe) : consultation des résultats, bulletins, devoirs, documents de cours et horaire.

5. Authentification et sécurité d'accès

- **Mot de passe** : haché avec bcrypt (coût 12). Comparaison à durée constante (ne révèle pas l'existence d'un compte).
- **2FA par courriel** : code à usage unique (haché, expirant) pour les comptes parent avant l'ouverture de session.
- **Connexion biométrique (WebAuthn / passkeys)** : Face ID, Touch ID, Windows Hello ou empreinte. Vérification biométrique *obligatoire*, passkeys découvrables (connexion sans saisir le courriel), autofill. La donnée biométrique ne quitte jamais l'appareil.
- **Google Identity** : connexion / inscription via compte Google.
- **Anti-force brute** : verrouillage par identifiant (en plus de la limite par IP) sur toutes les connexions, y compris les 6 espaces admin.
- **Sessions** : cookie `HttpOnly` + `Secure` + `SameSite=Lax`, identifiant régénéré à la connexion (anti-fixation), stockage MySQL.

6. Modules fonctionnels

6.1 Inscriptions

Le parent (ou l'élève lui-même, si autorisé) soumet un dossier : élève (nom, prénom, sexe, date de naissance — **âge minimum 2 ans**, niveau) et informations du responsable (profession choisie dans une liste d'une soixantaine de métiers + « Autre »). Si le paiement à l'inscription est exigé, le dossier reste « en attente de paiement » (invisible au secrétariat) jusqu'au règlement ; sinon il est transmis directement. Pour un compte élève, le nom/prénom de l'élève sont repris du compte et verrouillés, et les champs du responsable sont à remplir par l'élève. Un assistant IA aide le secrétariat à rédiger le message au parent.

6.2 Paiements

Frais à tarif configurable (inscription, relevé, duplicata) + frais divers. Paiement en ligne : **MonCash, NatCash, KashPay** et **carte Visa/Mastercard** via l'agrégateur.

- Confirmation serveur à serveur (jamais sur la foi du navigateur) ; webhook + reconciliation périodique comme filets de sécurité.
- Anti-rejeu (transaction unique), verrou anti double-clic, redirection restreinte au domaine autorisé (HTTPS).
- Paiement manuel avec preuve possible pour les relevés/duplicatas.

6.3 Notes et bulletins

Saisie par matière et trimestre, barème configurable par note, moyenne pondérée, classement, seuil de réussite. La publication génère les **bulletins PDF** (format paysage traditionnel, logo de l'école, appréciation IA), envoyés par courriel et disponibles en ligne.

6.4 Présence

Pointage quotidien par scan du badge QR (entrée/sortie) avec notification au parent.

6.5 Espaces publics & Économat

Vlog, Académie, Corps enseignant, À propos, Examens passés (avec tuteur IA), formulaire de contact. L'**Économat** gère la file des paiements, les tarifs, les frais divers et les acquittements de scolarité.

7. Contrôle d'accès du propriétaire

Depuis son espace, le propriétaire active ou désactive des fonctions sensibles (table `permissions`). Chaque interrupteur s'applique côté serveur.

Interrupteur	Effet	Défaut
Matières & coefficients	Autorise la direction à gérer les matières	Autorisé
Saisie des notes	Autorise le secrétariat à saisir/valider les notes	Autorisé
Publication des notes	Autorise la publication/retrait des bulletins	Autorisé
Paiement à l'inscription	Exige le paiement en ligne avant transmission au secrétariat ; désactivé = inscription sans paiement	Autorisé
Auto-inscription des élèves	Permet à un élève de créer son compte et s'inscrire	Désactivé

Le propriétaire dispose aussi d'un **Rapport mensuel** : pour un mois donné (depuis la création du système), résumé des actions (inscriptions, paiements, documents), activité par agent (direction/secrétariat) et connexions du personnel — exportable en **Excel/CSV**.

8. Base de données

MySQL, ≈38 tables. Principales familles :

Domaine	Tables
Comptes & auth	utilisateurs, parents, codes_verification, webauthn_credentials, sessions
Scolarité	eleves, inscriptions, documents_soumis, demandes_anciens_eleves
Notes & bulletins	matieres, notes, publications_notes, documents_officiels, enseignant_matieres
Enseignement	professeurs, profs_admins, devoirs, documents_cours, horaires_cours
Paiements	tarifs, frais_divers, paiements, acquittements_scolarite, economat_admins
Présence	presences
Contenus publics	ecole, vlog_admins, publications_vlog, apropos_admins, apropos_sections, academie_admins, academie_sections, examens_admins, examens
Administration	permissions, messages_contact, historique_statut, journal_securite

Schéma complet dans `backend/db/schema.sql` ; 39 migrations incrémentales dans `backend/db/migrations/`.

9. API REST

Environ **188 endpoints** en 16 modules, préfixés par `/api`. Chaque route est protégée côté serveur (`requireAuth`, `requireRole`, `requireEleve`, `requireEconomatAdmin`…) et, pour les endpoints sensibles, par une limitation de débit. Les routes publiques (listes et photos des espaces vlog/ académie/ professeurs/examens, logo, marque) ne servent que du contenu public.

10. Sécurité & tests d'intrusion

- **Compte base de données à privilèges limités** : l'application se connecte avec un compte `lekol_app` restreint à SELECT/INSERT/UPDATE/DELETE. CREATE/DROP/ALTER refusés — une faille éventuelle ne peut pas altérer la structure.
- **En-têtes** : HSTS, CSP stricte (`object-src 'none'` , `frame-ancestors 'self'`), nosniff, X-Frame-Options, Referrer-Policy, Permissions-Policy, pas de X-Powered-By.
- **Injections** : requêtes SQL paramétrées partout ; identifiants d'URL validés (entier positif) ; corps JSON limité ; pas de `eval` / `exec` .
- **Téléversements** : vérification du vrai type par octets d'en-tête (magic bytes), limite de taille, protection contre le path traversal.
- **Paiements** : confirmation serveur à serveur, anti-rejeu ; le webhook ne fait jamais confiance à son contenu.
- **IA** : clés côté serveur, consignes de sécurité, réponses vérifiées, noms des élèves/parents non transmis, appels bornés.
- **Journal de sécurité + alertes courriel** (connexions, verrouillages, changements sensibles, anomalies de paiement) ; purge automatique (journal 180 j, codes 7 j).
- **Dépendances** : audit à 0 vulnérabilité (backend et frontend).

> **Test d'intrusion (red-team) :** chaque vecteur d'attaque a échoué — session forgée (401), escalade de rôle / mass-assignment (ignorée), IDOR (403), injection SQL (aucune fuite), path traversal (400), webhook de paiement forgé (ignoré, re-vérifié côté serveur), validation sans autorisation (401), assertion WebAuthn forgée (401), limitation de débit (429). Aucune vulnérabilité critique.

11. Déploiement

Déploiement sur **Railway** en un seul service Docker (build du frontend puis API + frontend servis ensemble). Base MySQL gérée par Railway ; fichiers (logos, bulletins, preuves) sur un **volume persistant**. Serveur exécuté sous l'utilisateur `node`.

Variables d'environnement principales

```
DB_HOST, DB_PORT, DB_USER, DB_NAME, DB_PASSWORD   base de données (compte lekol_app)
SESSION_SECRET                                     secret des sessions (obligatoire en prod)
RESEND_API_KEY, MAIL_FROM                          envoi de courriel
TEXTBELT_API_KEY                                   envoi de SMS
PLOPPLOP_CLIENT_ID, PLOPPLOP_METHODE_CARTE         paiements mobiles / carte
GROQ_API_KEY, GROQ_MODEL                           assistant IA
GOOGLE_CLIENT_ID                                   connexion Google
WEBAUTHN_RP_ID, WEBAUTHN_ORIGINS                   connexion biométrique
ALERTE_COURRIEL                                    destinataires des alertes de sécurité
MAINTENANCE_MODE                                   page de maintenance (true/false)
```

Base neuve : exécuter `schema.sql`. Base existante : appliquer les nouvelles migrations `db/migrations/*.sql`. `MAINTENANCE_MODE=true` ferme le site pendant une opération (sans redéploiement du code).

12. Tests et qualité

Tests automatisés avec `node:test`, exécutés sans base ni réseau grâce à des doublures : moyennes pondérées, année scolaire, regroupement des bulletins, et surtout le service de paiement (refus d'accès, double validation, génération de documents, régénération, anti-rejeu, transmission des dossiers). Le build frontend (Vite) et la suite de tests sont vérifiés à chaque évolution.

13. Crédits

> **Lekòl Konekte** a été conçue et développée par **Plaisival Steven**.

- 🏫 Plateforme : https://lekolkonekte.online
- 👤 Portfolio : https://plaisivalsteven.online
- 💻 GitHub : https://github.com/Steven0p
- 💼 LinkedIn : https://www.linkedin.com/in/steven-plaisival-08a996428/

Document mis à jour le 4 octobre 2026 — © 2026 Lekòl Konekte. Tous droits réservés.